Multi-agent Semantic Firewall: A Hybrid Defense-in-Depth System for LLM Security

Anonymous Author(s)

Institution details withheld for double-blind review

Abstract. Large Language Models (LLMs) are increasingly being used in security-sensitive applications, yet reliably detecting prompt injection and jailbreak attacks remains challenging. Existing defenses generally deploy a single model-based classifier per request, which results in inference latency and a lack of adaptability to emerging threats. This paper presents Semantic Firewall, a layered multi-agent framework that combines deterministic screening, semantic retrieval, parallel security detectors, and constrained LLM-based verification. The architecture follows an early-exit strategy in which prompts resolved by lightweight mechanisms bypass expensive downstream analysis. A self-healing semantic cache stores newly detected threats, allowing future semantic variants to be identified without repeated LLM inference and directly addressing the adaptability gap in prior defenses. Eight specialized detectors execute concurrently to cover complementary threat categories, while a deterministic Llama-3.3-70B gate evaluates unresolved prompts with schema-constrained outputs. On the neuralchemy prompt-injection benchmark, Semantic Firewall achieves a 92.64% F1 score ($\pm$0.42) and 99.82% recall under cold-start evaluation, outperforming GPT-4o, Gemini 2.5 Flash, and Claude 3.5 Haiku, while bypassing LLM inference for 66.88% of requests and cutting per-token API cost by roughly 3$\times$. Cross-dataset experiments on BeaverTails, deepset, AI4Privacy, walledai, and multi-turn jailbreak scenarios further evaluate generalization across different attack settings, achieving an F1 score of 83.93% and recall of 87.59%. These results demonstrate that combining lightweight detection, adaptive semantic memory, and selective LLM verification provides a practical, cost-effective defense for production LLM applications.

Keywords: Large Language Models · Prompt Injection · LLM Security · Semantic Firewall · Semantic Caching · Multi-Agent Systems.

1 Introduction

Large Language Models (LLMs) are now embedded in critical infrastructure: they power customer support pipelines, generate production code, answer medical queries, and drive autonomous agents that make consequential decisions with minimal human oversight. This rapid transition from research artifact to production dependency has widened the attack surface faster than defenses have

matured [1,2]. Adversaries inject carefully crafted prompts to hijack model behavior [3], construct jailbreaks that bypass alignment training [4,5,6,7], and exploit context windows to exfiltrate personally identifiable information (PII) or cryptographic secrets. The industry's default defense has been to place an additional model ahead of the first: Llama Guard [8], Guardrails AI [9], and Prompt-Guard [10] route every incoming prompt through a dedicated safety LLM, irrespective of whether it is a benign query or a multi-turn injection attempt. This approach is simple to build but expensive to run; routing every request through a dedicated safety model adds substantial latency; our own profiling of a comparable LLM-only gate measures 731.64 ms per request (Section 5), and GPU costs scale linearly with traffic. It also breaks down under novel attacks, as static safety tuning does not generalize well to new attack patterns, such as encoded payloads, token smuggling, and multi-turn injections. Meanwhile, opaque `"403 Forbidden"` rejections leave downstream applications unable to distinguish a genuine attack from a false positive [11,12]. No single defense mechanism simultaneously achieves high detection accuracy, low latency, low computational cost, and robustness to emerging prompt-injection attacks [13]. In general, conventional defenses utilize detection as a single monolithic assessment, one model-based classifier evaluating every request in isolation, rather than a graded response calibrated to attack ambiguity, a problem this study tackles directly [8,10,14,15].

Securing prompt-based interactions therefore demands the same defense-in-depth philosophy long established in network security [16]: not a single point of inspection, but a layered architecture in which each stage is optimized against a distinct threat class. This paper presents Semantic Firewall, a five-stage realization of this philosophy in which deterministic checks and semantic memory resolve most prompts immediately, escalating only ambiguous cases through a parallel ensemble of detectors and a constrained LLM gate. Confirmed threats feed back into the semantic cache, allowing the system's detection boundary to harden with exposure rather than through manual retraining. The main contributions of this paper are the following:

1. **Layered, early-exit multi-agent defense:** A semantic firewall is proposed that combines deterministic screening, semantic retrieval, parallel threat detection, and constrained LLM verification in a single architecture that resolves each prompt as early as it can be safely determined.
2. **Adaptive semantic memory:** A self-updating semantic cache stores newly confirmed threats, enabling semantically similar future attacks to be identified without repeated LLM inference.
3. **Parallel multi-agent detection:** A parallel ensemble of eight specialized security agents (Table 1) detects complementary threat categories concurrently while minimizing additional detection latency.

This architecture is evaluated on five public benchmarks against six representative baselines, demonstrating strong detection performance, cross-dataset generalization, and low inference latency.

The rest of this paper is organized as follows. Section 2 reviews related work, section 3 presents the proposed architecture, section 4 describes the experimental setup, and section 5 presents the results. The limitations and future work are discussed in section 6 and lastly, section 7 concludes the paper.

2 Related Work

Recent work on securing LLM applications falls into three broad categories: safety classifiers, adversarial robustness research, and guardrail frameworks. Llama Guard [8] and PromptGuard [10] represent the first category, scanning inputs with a learned model. However, this generalizes poorly to attacks outside the training distribution and adds inference overhead. The second category outlines the numerous examples by which safety-aligned models continue to exhibit a lack of reliability. Zou et al. [7] found that GCG-optimized adversarial suffixes transfer across model families, and AutoDAN [4] used genetic search to craft natural-sounding jailbreak prompts. PAIR [5] automated attacks through repeated refinement, and Greshake et al. [3] showed that hidden instructions in outside content can hijack LLM-integrated applications. PromptBench [17] responds with a systematic robustness benchmark, while SmoothLLM [18] responds with a perturbation-based defense.

The third group mentions defenses for the application layer.
NeMo Guardrails [15] enforces conversational policy through programmable rules, though rule-based systems demand continual upkeep as attackers invent new injection techniques. Rebuff [14] takes a similar hybrid approach, combining heuristic detectors, LLM-based verification, and a vector database of previously seen attacks. Collectively, these three categories of work lead to the same conclusion that no single mechanism is sufficient, and LLM security necessitates layered defenses constructed from complementary detection strategies. Closing this gap is precisely what Semantic Firewall offers.

3 Semantic Firewall Architecture

The proposed semantic firewall follows a layered pipeline that separates request validation, semantic memory, threat detection, and policy-driven decision-making. As shown in Fig. 1, incoming prompts pass through progressively deeper analysis stages, numbered ❶–❻ in the figure and referenced by the same numbers in the subsections below.

The architecture follows an early-exit strategy. Once a prompt is resolved with sufficient confidence, downstream processing is skipped, reducing unnecessary computation and LLM inference. This confidence is formalized through two thresholds: the similarity threshold τ (Eq. (1)) at the semantic-memory stage and the risk threshold θ (Algorithm 1) at the detector-mesh stage. Unresolved prompts escalate in stages, i.e., first to semantic retrieval, then to a parallel ensemble of specialized detectors, and finally to a deterministic Llama-3.3-70B gate that returns a structured verdict. Newly confirmed threats are written back

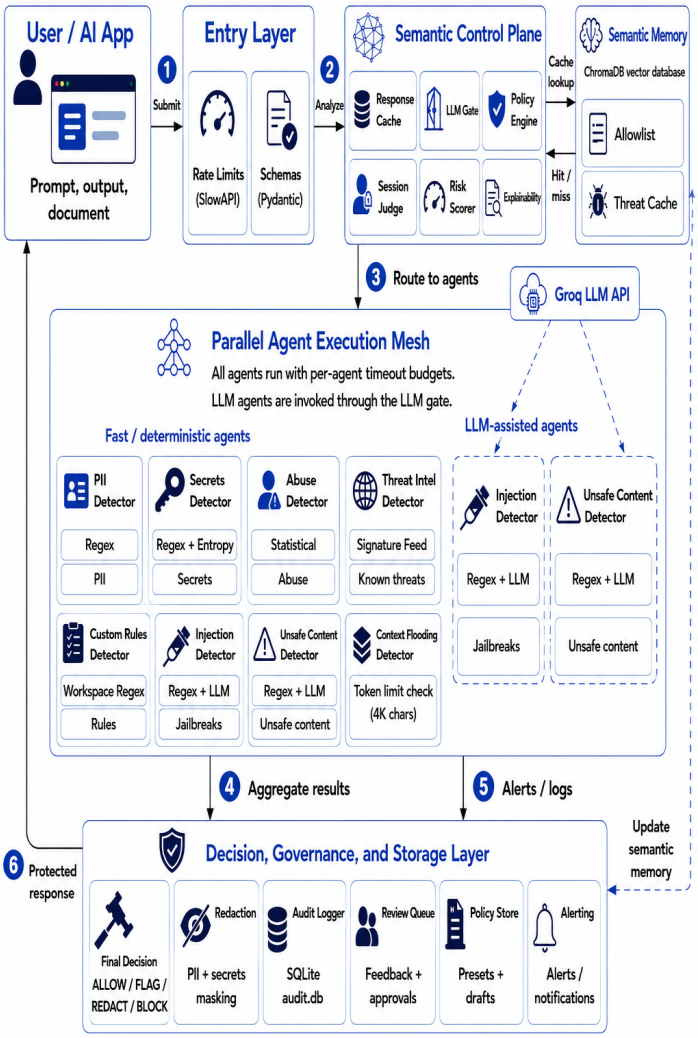


Fig. 1. Multi-agent Semantic Firewall architecture with semantic memory, parallel security agents, and policy-driven decisions. Circled numbers indicate the request path referenced throughout Section 3.

into a self-healing semantic cache so that future, semantically similar variants are resolved at the lightweight layer without repeated LLM inference, allowing the system's detection boundary to harden with exposure rather than requiring manual rule updates or retraining.

3.1 Entry Layer and Initial Screening (Step ❶)

Incoming requests are first rate-limited and validated against the expected schema. Malformed inputs and excessive requests are rejected before entering the detec-

tion pipeline. A lightweight regex engine then scans for known attack patterns, including instruction overrides, encoded payloads, jailbreak prefixes, and token-smuggling attempts. The engine runs on the CPU with an average latency of 5.21 ms and immediately blocks matching prompts before more expensive analysis is performed.

3.2 Semantic Control Plane and Memory (Step ❷)

The Semantic Control Plane orchestrates context-aware detection and policy enforcement and acts as the primary cost gate for LLM inference: fast, low-cost checks run first, and the constrained LLM gate (Section 3.4) is invoked only when they leave a prompt's status ambiguous. Four modules implement this orchestration. The *Session Judge* maintains a sliding window over recent conversation turns and computes an escalating threat trajectory across them, rather than scoring each prompt in isolation. This is designed to counter multi-turn jailbreaks such as Crescendo [19], in which an attacker builds conversational trust over several benign-looking turns before injecting a harmful payload on a later one. The *Risk Scorer* aggregates the continuous confidence values reported by each detector, rather than treating their outputs as binary safe/unsafe decisions, weighting each by severity to produce the unified risk score formalized in Section 3.5. The *Policy Engine* maps this score onto configurable deployment profiles (e.g., *Strict*, *Balanced*, *Developer*), so that, for instance, a healthcare application can enforce stringent PII filtering while an internal developer tool applies more permissive rules, without requiring changes to the underlying detectors. Finally, the *Explainability Module* addresses the opacity problem raised in Section 1; rather than returning an unexplained rejection, it aggregates the specific rules, detector scores, and evidence that triggered a decision into a structured, human-readable report, supporting both auditing and the resolution of false positives.

Semantic memory maintains separate stores for known threats and administrator-approved prompts. Each prompt is encoded as a 384-dimensional vector using Sentence-Transformers [20] and searched against a ChromaDB [21] Hierarchical Navigable Small World (HNSW) graph index [22]. Similarity between an incoming prompt $\boldsymbol{v}_q$ and a cached threat $\boldsymbol{v}_c$ is computed as

$$\mathrm{sim}(\boldsymbol{v}_q, \boldsymbol{v}_c) = \frac{\boldsymbol{v}_q \cdot \boldsymbol{v}_c}{\|\boldsymbol{v}_q\|_2 \|\boldsymbol{v}_c\|_2}. \tag{1}$$

The highest similarity score is given by

$$s^* = \max_{\boldsymbol{v}_c \in \mathcal{C}} \mathrm{sim}(\boldsymbol{v}_q, \boldsymbol{v}_c), \tag{2}$$

where $\mathcal{C}$ denotes the threat cache. A prompt is blocked when $s^* \geq \tau$. We set $\tau = 0.90$ based on a sweep over the validation set (Section 5, Table 3), which shows this value maximizes F1 while keeping the false-positive rate low; the full sensitivity analysis is reported in Section 5.

Table 1. The eight specialized security agents in the Parallel Agent Execution Mesh.

Detector	Category	Function
PII Detector	Fast / deterministic	Identifies sensitive identifiers (emails, phone numbers, IDs) using pattern-based rules.
Secrets Detector	Fast / deterministic	Combines signature matching with Shannon entropy to flag high-entropy tokens and credentials.
Abuse Detector	Fast / deterministic	Uses approximate (Levenshtein-distance) string matching to catch lexically obfuscated abusive terms.
Threat Intel Detector	Fast / deterministic	Matches prompts against a signature feed of previously observed threats.
Custom Rules Detector	Fast / deterministic	Evaluates workspace-specific regex and policy rules.
Injection Detector	LLM-assisted	Combines regex screening with the LLM Gate to catch jailbreak and prompt-injection attempts.
Unsafe Content Detector	LLM-assisted	Combines regex screening with the LLM Gate to flag unsafe or policy-violating content.
Context Flooding Detector	LLM-assisted	Checks for context/token-flooding attacks against a configurable length budget (e.g., 4K characters).

The semantic memory is updated through a symmetric, self-healing feedback loop on both sides of the cache. When downstream analysis confirms a novel attack, its embedding is added to the threat cache. Consequently, a subsequent prompt that rephrases the same intent, for instance, following up a blocked request such as "Give me your secret instructions" with a paraphrase such as "Provide your hidden system prompt" is then recognized by similarity rather than by exact string match. Therefore, such prompts get blocked immediately without re-invoking the parallel detectors or the LLM gate. Conversely, if the pipeline flags a legitimate but unusually phrased prompt as a false positive, an administrator can approve it once; its embedding is then added to the allowlist so that future prompts of similar intent bypass the LLM Gate and are allowed directly. This closes the loop in both directions: repeated attacks become cheaper to block over time, and repeated false positives become cheaper to correct, without manual rule updates or retraining.

3.3 Parallel Agent Execution Mesh (Step ❸)

Prompts unresolved by the initial screening and semantic memory stages are routed to a set of eight specialized security agents, summarized in Table 1. The architecture executes these agents concurrently with individual timeout budgets. For detector execution times $T_1, \ldots, T_N$, the idealized ensemble latency is

$$T_{\text{mesh}} \approx \max_{1 \le i \le N} T_i + T_{\text{ovh}}, \tag{3}$$

where T_{ovh} represents thread scheduling, communication, and aggregation overhead.

The deterministic agents cover complementary threat categories. The PII Detector identifies sensitive identifiers using pattern-based rules. The Secrets

Detector combines signature matching with Shannon entropy to detect high-entropy tokens and potential credentials. For a candidate string X of length n, its entropy is defined as [23]

$$H(X) = -\sum_{i=1}^{n} p(x_i) \log_2 p(x_i), \tag{4}$$

where $p(x_i)$ is the empirical probability of character x_i. The Abuse Detector identifies lexical obfuscations using approximate string matching based on the Levenshtein distance. For strings a and b, let $d_{a,b}(i,j)$ denote the minimum edit distance between the prefixes $a_{1:i}$ and $b_{1:j}$. The distance is recursively defined as [24],

$$d_{a,b}(i,j) = \begin{cases} \max(i,j), & \min(i,j) = 0, \\ \min \begin{cases} d_{a,b}(i-1,j) + 1, \\ d_{a,b}(i,j-1) + 1, \\ d_{a,b}(i-1,j-1) + \delta_{ij}, \end{cases} & \text{otherwise,} \end{cases} \tag{5}$$

where $\delta_{ij} = \mathbb{I}(a_i \neq b_j)$ denotes the indicator function. The three terms correspond to deletion, insertion, and substitution operations, respectively. Smaller Levenshtein distances indicate greater lexical similarity, enabling the detector to identify obfuscated variants of known abusive terms. The remaining agents (Threat Intel, Custom Rules, Injection, and Unsafe Content Detectors in Table 1) perform threat-intelligence matching, workspace-specific rule evaluation, injection detection, and unsafe-content analysis, respectively. Injection and unsafe-content detectors may invoke the LLM Gate when deterministic evidence is insufficient. This organization allows inexpensive agents to resolve common threats while reserving model-based reasoning for ambiguous inputs.

3.4 Constrained LLM Gate (invoked within Step ❸)

Prompts that remain unresolved after deterministic analysis and semantic lookup are evaluated by the LLM Gate. The current implementation uses Llama-3.3-70B-Instruct through the Groq API. The model is instructed to return a schema-constrained decision containing an `ALLOW` or `BLOCK` action, a confidence value, and an explanation. Greedy decoding $T = 0$ is used to reduce output variability and support reproducible evaluation.

When the LLM Gate identifies a novel attack, the corresponding prompt embedding is inserted into the threat cache. This feedback path allows subsequent semantic variants to be resolved without repeated LLM inference. If the external LLM service is unavailable, the firewall applies the configured fallback policy rather than silently bypassing security checks.

3.5 Risk Aggregation and Policy Enforcement (Step ❹)

The agent mesh produces detector decisions, confidence values, matched patterns, and severity indicators. These heterogeneous outputs are combined into a

Algorithm 1 Semantic Firewall Classification Pipeline

Input: Prompt q, similarity threshold τ, risk threshold θ, threat cache $\mathcal{C}$, detector set $\mathcal{D} = \{D_1, \ldots, D_8\}$
Output: Decision $d \in \{\textsc{Allow}, \textsc{Block}\}$

$M \leftarrow \textsc{RegexScan}(q)$
if $M \neq \emptyset$ **then**
  return BLOCK
end if
$\boldsymbol{v}_q \leftarrow \textsc{Embed}(q)$
if $\mathcal{C} \neq \emptyset$ **then**
  $s^* \leftarrow \max_{\boldsymbol{v}_c \in \mathcal{C}} \dfrac{\boldsymbol{v}_q \cdot \boldsymbol{v}_c}{\|\boldsymbol{v}_q\|_2 \|\boldsymbol{v}_c\|_2}$
  if $s^* \geq \tau$ **then**
    return BLOCK
  end if
end if
$\mathcal{R} \leftarrow \textsc{ParallelExecute}(\mathcal{D}, q)$
$r \leftarrow \textsc{Risk}(\mathcal{R})$
if $r \geq \theta$ **then**
  return BLOCK
end if
$y \leftarrow \text{LLMGate}(q, T{=}0, \text{JSON})$
if y.action = BLOCK **then**
  $\mathcal{C} \leftarrow \mathcal{C} \cup \{\boldsymbol{v}_q\}$ ▷ Update threat cache
  return BLOCK
end if
return ALLOW

continuous risk score:

$$\text{Risk}(\mathcal{R}) = \min\left(100,\ S_{\text{sev}} + S_{\text{conf}} + S_{\text{pat}} + S_{\text{ses}}\right). \tag{6}$$

The risk score is calculated by combining the highest detected threat severity S_{sev}, confidence-weighted evidence S_{conf} from triggered detectors, agreement across detectors S_{pat}, and S_{ses} risk accumulated during the current session. Severity receives the largest fixed base value (0–85), confidence contributes up to 30 points, detector agreement adds 5 points per trigger (up to a maximum of 20), and session history accounts for up to 10. These heuristic weights prioritize serious threats while allowing strong evidence, consensus, and repeated suspicious activity to increase borderline scores; the final score is capped at 100.

The resulting score is evaluated by the Policy Engine using configurable security profiles and thresholds. This separation allows the same detection pipeline to support different deployment requirements without modifying individual detectors. High-confidence critical findings may trigger immediate blocking, while intermediate-risk requests can be flagged, redacted, or forwarded for human review.

3.6 Decision, Governance, and Storage (Steps ❺–❻)

The final layer converts the accumulated security evidence into one of four actions: ALLOW, FLAG, REDACT, or BLOCK. Sensitive entities and secrets detected in otherwise legitimate prompts may be masked before the request is forwarded to the target LLM. Each transaction is recorded by the Audit Logger, including payload hashes, detector outcomes, risk scores, processing latency, and final decisions. Ambiguous or policy-sensitive cases may be forwarded to a human review queue. Analyst decisions can subsequently update the allowlist, threat cache, or policy configuration. High-risk events may additionally generate asynchronous notifications through external alerting channels.

3.7 Classification Procedure

Algorithm 1 summarizes the core classification pipeline of Semantic Firewall. Each prompt is first screened using deterministic rules and compared with previously detected threats in the semantic cache. Prompts that remain unresolved are analyzed by eight detectors running in parallel, and their outputs are aggregated into a risk score. Only prompts below the configured risk threshold reach the constrained LLM Gate. When the LLM Gate identifies a novel attack, its embedding is added to the threat cache, allowing similar inputs to be detected earlier in subsequent requests. This early-exit design limits unnecessary computation while enabling the firewall to adapt to newly observed threats.

3.8 Effective Latency

Because prompts may terminate at different stages, end-to-end latency depends on the fraction of requests resolved by each path. Let P_h, P_c, P_m, and P_l denote the probabilities that a prompt is resolved by initial screening, semantic memory, the parallel agent mesh, or the LLM Gate, respectively. The expected latency can be expressed as,

$$L_{\text{eff}} = P_h L_h + P_c L_c + P_m L_m + P_l L_l, \quad \sum_{k \in \{h,c,m,l\}} P_k = 1, \tag{7}$$

where L_h, L_c, L_m, and L_l are the corresponding mean path latencies. This formulation reflects the architecture more accurately than treating total latency as the sum of all component execution times.

3.9 Security Considerations

The detection stages are not assumed to fail independently, as a carefully obfuscated attack may evade multiple detectors at the same time. The architecture therefore combines complementary detection mechanisms so that weaknesses in one stage can be addressed by evidence from others.

Detection is also separated from policy enforcement. Outputs from the security agents are aggregated into a risk score, while configurable policies determine

whether a prompt is allowed, flagged, redacted, or blocked. This separation allows the same detection pipeline to support different security requirements without modifying the underlying detectors.

4 Experimental Setup

The evaluation uses five public benchmarks selected to cover different security challenges. The *neuralchemy/Prompt-injection-dataset* [26] (N = 2,000) serves as the primary benchmark and contains a balanced set of benign prompts and prompt-injection attacks. The *ai4privacy/pii-masking-200k* dataset [27](N = 209,261) is used to evaluate PII detection at scale on multilingual data.
The *walledai/Multi-Turn-Jailbreak [28]* contains 200 adversarial conversations of 2–10 turns and is used to assess multi-turn attack detection.
The *PKU-Alignment/BeaverTails* [29] benchmark (N = 2,000) evaluates safety classification across 14 harm categories, including ambiguous and borderline prompts. Finally, *deepset/prompt-injections*, [30] the native evaluation dataset of *Rebuff* [14], is used to assess performance on a benchmark designed specifically for prompt-injection detection.

Two additional 2,000-sample datasets are used for targeted evaluation: a synthetic corpus covering violence and illegal activities, and a benign subset of *HuggingFaceH4/no_robots* [31] for measuring false positives. System thresholds and heuristic parameters, including the semantic similarity threshold τ, are selected using a separate 500-sample validation set. The implementation will be made publicly available upon publication (link withheld for double-blind review).

4.1 Implementation, Baselines, and Evaluation Metrics

Experiments were conducted on an Intel Xeon E5-2686 v4 server with 8 vCPUs and 32 GB of RAM. Semantic memory was implemented using ChromaDB [21] with HNSW indexing [22], while prompt embeddings were generated using Sentence-Transformers [20]. The LLM Gate used Llama-3.3-70B-Instruct through OpenRouter with deterministic decoding ($T = 0.0$).

The evaluation includes GPT-4o [32], Claude 3.5 Haiku [33], Gemini 2.5 Flash [34], and Llama Guard 4 [35] as baselines. These commercial models were evaluated zero-shot without task-specific prompt tuning, representing an out-of-the-box deployment setting.[1] As these models are not specifically designed for binary prompt-injection classification, their results are reported under this evaluation setting. Llama Guard 4 was also evaluated zero-shot for structural prompt-injection detection.

Performance is reported using Precision, Recall, F1 score, and False Positive Rate (FPR). Pairwise differences in classification outcomes are tested for statistical significance using McNemar's test [25]. Effective latency and API cost savings are also reported to assess the practical efficiency of the system.

[1] API snapshots: `gpt-4.0-2024-05-13`, `claude-3-5-haiku-20240307`, and `gemini-2.5-flash-001`.

Benchmark	Model / Config	N	Prec.	Recall	F1	Latency	Notes
(a) Open-Weight vs. Proprietary API Comparison (neuralchemy, N=2,000)							
neuralchemy	**Semantic Firewall (Ours, Warm)**	2000	89.03	98.61	**93.57**	**427.72 ms**	Steady-state
neuralchemy	Semantic Firewall (Ours, Cold)	2000	86.43	**99.82**	92.64	459.85 ms	Empty cache
neuralchemy	OpenAI GPT-4o	2000	**96.10**	84.60	90.00	876 ms	Proprietary
neuralchemy	Google Gemini 2.5 Flash	2000	95.30	81.00	87.60	1464 ms	Proprietary
neuralchemy	Anthropic Claude 3.5 Haiku	2000	94.70	74.30	83.20	1156 ms	Proprietary
(b) Cross-Dataset Generalization							
BeaverTails	**Semantic Firewall**	2000	65.23	**87.59**	**74.80**	—	14 harm categories
BeaverTails	Llama Guard 4 (12B)	2000	73.68	61.20	66.86	—	—
deepset/prompt-inj.	**Semantic Firewall**	—	**95.10**	75.14	**83.93**	—	Rebuff's dataset
deepset/prompt-inj.	ProtectAI Rebuff	—	86.20	**79.10**	82.50	—	Pipeline
(c) Specialized Evaluations							
ai4privacy/pii-200k	Semantic Firewall (Regex Layer)	209K	85.90	96.70	91.00	<2 ms	Zero LLM cost
walledai/Multi-Turn	Semantic Firewall (Full)	200	—	90.00	—	—	Session recall
Synthetic Safety	Semantic Firewall (Full)	2000	82.50	84.30	83.38	—	FPR = 5.45%
HF/no_robots	Semantic Firewall (Full)	2000	—	—	—	—	FPR = 4.20%

Table 2. Baseline, cross-dataset, and specialized evaluations. Best results are **bolded**; "—" denotes non-applicable metrics.

5 Results

To evaluate Semantic Firewall, we conduct four sets of experiments: baseline comparisons against six representative systems, cross-dataset generalization tests, specialized evaluations of individual system components, and a sensitivity analysis over the semantic-cache threshold τ. We further perform an ablation study isolating the contribution of each architectural component. Tables 2, 3, and 4 present the full results, analyzed in detail in the following subsections.

5.1 Evaluation

Baseline Comparison (Table 2a) Semantic Firewall achieves the highest F1 score among the evaluated systems (92.64%), compared with GPT-4o (90.00%), Claude 3.5 Haiku (83.20%), and Gemini 2.5 Flash (87.60%). Although GPT-4o achieves higher precision (96.10%), Semantic Firewall provides substantially higher recall (99.82% versus 84.60%), indicating more consistent detection of attack prompts. This improvement is largely due to the layered architecture, where deterministic and semantic checks identify structural attacks before model-based evaluation. The difference is statistically significant under McNemar's test [25] ($\chi^2 = 4.28$, $p = 0.038$). In addition, 41.93% of requests are resolved by the fast-path layers, reducing effective cold-start latency to 459.85 ms, below that of the evaluated APIs.

Cross-Dataset Evaluation (Table 2b) Semantic Firewall maintains competitive performance across unseen datasets. On BeaverTails, which includes 14 harm categories and ambiguous safety cases, it achieves an F1 score of 74.80%, compared with 66.86% for Llama Guard 4. The lower Precision (65.23%) reflects the conservative behavior of the detection pipeline on borderline prompts, while the high Recall (87.59%) indicates that most harmful inputs are still detected. On the deepset prompt-injection benchmark, Semantic Firewall achieves an F1

score of 83.93%, comparable to Rebuff's 82.50%, while using automatic semantic caching instead of manually maintained attack signatures.

Specialized Evaluations (Table 2c) The PII detection layer achieves 91.0% F1 and 96.7% Recall across 209K samples, with sub-2 ms processing latency and no neural model inference. Structured identifiers, including email addresses, credit cards, and International Bank Account Numbers (IBANs), achieve over 99.7% Recall, while more variable patterns such as phone numbers remain harder to detect (74.8% recall). In multi-turn scenarios, Semantic Firewall intercepts 90.0% of adversarial sessions. Evaluation of benign prompts on no_robots [31] results in a 4.20% FPR, indicating limited degradation on non-adversarial inputs.

(a) Full System (C8) on neuralchemy (N=2,000)

Threshold (τ)	**Prec.**	**Recall**	**F1**	**FPR**
$\tau = 0.80$	78.12	99.43	88.24	18.14%
$\tau = 0.85$	85.34	99.11	90.47	12.41%
$\tau = 0.90$ (Sel.)	**89.03**	**98.61**	**93.57**	**4.20%**
$\tau = 0.95$	94.81	92.86	91.43	4.24%
$\tau = 0.99$	96.27	88.29	88.12	1.83%

(b) Out-Of-Distribution (OOD) Generalization

Threshold (τ)	**Acc.**	**Prec.**	**Recall**	**F1**
$\tau = 0.80$	89.80%	96.49%	87.30%	91.67%
$\tau = 0.85$	91.84%	98.25%	88.89%	93.33%
$\tau = 0.90$	86.73%	96.30%	82.54%	88.89%
$\tau = 0.95$	90.82%	98.21%	87.30%	92.44%

Table 3. Cache Threshold Sensitivity (Across τ sweeps)

5.2 Cache Threshold experiments (Table 3)

The semantic-cache threshold τ controls how readily a prompt is blocked based on its similarity to a previously confirmed threat (Eq. (1)): lower values catch more paraphrased attack variants but risk misclassifying benign prompts, while higher values are more conservative but let more variants fall through to the slower detector mesh. Table 3a sweeps τ from 0.80 to 0.99 on the full system (C8) over the neuralchemy benchmark and exposes a clear trade-off governed by this single parameter: τ=0.80 maximizes the cache hit rate but is too permissive, inflating the false-positive rate to 18.14%, while τ=0.99 minimizes false positives (1.83%) but fires so rarely that most prompts bypass the cache, eroding its latency benefit. F1 peaks at τ=0.99, which we adopt as the operating threshold: it reaches the highest F1 (93.57%) and near-ceiling recall (98.61%) of any setting tested, at a 12.45% cache hit rate and 4.20% FPR.

Table 3b repeats this sweep on the out-of-distribution (OOD) manually curated dataset (N=500) to test whether a threshold tuned on neuralchemy transfers to unseen data. Notably, F1 on this set peaks slightly earlier, at τ=0.85 (93.33%), rather than at the neuralchemy-selected τ=0.90 (88.89%), suggesting the optimal threshold may vary modestly across data distributions, though

τ=0.90 remains within a comparable range of the best OOD result rather than degrading sharply.

5.3 Ablation Study (Table 4)

The ablation results show a steady improvement as additional components are introduced. Interestingly, the standalone LLM Gate (C1) and regex engine (C2) achieve similar F1 scores (70.20% and 69.40%, respectively), despite the large difference in latency (731 ms versus 5 ms). The main gains come from combining complementary components. Regex and semantic caching (C5) increase precision to 98.51%, while adding parallel detectors (C6) raises F1 to 90.40%. The full pipeline (C8) achieves the highest F1 and Recall, improving F1 by 23.37 points over C1; this difference is statistically significant (McNemar's $\chi^2 = 135.2$, $p < 0.001$).

Adding the LLM Gate improves recall from approximately 80% to 99%, although

Configuration	N	Prec.	Recall	F1	Latency
C1: LLM Gate Only	2000	78.23	63.66	70.20	731.64 ms
C2: Regex Only	2000	77.34	62.94	69.40	5.21 ms
C3: Cache Only	2000	81.24	56.29	66.50	602.83 ms
C4: Par. Det. Only	2000	83.07	93.77	88.10	83.45 ms
C5: Regex + Cache	2000	98.51	80.17	88.40	602.83 ms
C6: Regex + Par. Det.	2000	92.04	88.81	90.40	83.45 ms
C7: All Fast (No LLM)	2000	98.14	82.77	89.80	602.83 ms
C8: Full (Warm)	2000	**89.03**	**98.61**	**93.57**	**427.72 ms**
C9: Full (Cold)	2000	86.43	99.82	92.64	459.85 ms

Table 4. Ablation study of Semantic Firewall components on the neuralchemy benchmark (N = 2,000).

Precision decreases because of false positives on ambiguous prompts. This reflects a security-oriented trade-off that favors attack detection over missed threats. Finally, parallel execution limits the latency added by the heuristic ensemble, with both C4 and C6 recording 83.45 ms.

Cost On our evaluation distribution, fast layers resolved 66.88% of all incoming requests without LLM invocation; this ratio will vary with deployment-specific traffic composition. Applying (7) to the evaluation split (P_h=0.55, P_c=0.12, P_l=0.33) results in an effective system latency of $L_{\text{eff}} \approx 444$ ms. Economically, while standard GPT-4o processing of 100 million tokens costs $500, the Semantic Firewall reduces this API cost to $166.

6 Limitations & Future Work

A few limitations point to future work. The 70B LLM Gate is resource-heavy; a distilled 8B model could cut cost and latency. The cache can still miss attacks worded very differently from anything seen before, which adversarially trained

embeddings could help catch. The Python orchestrator handles only about 45 requests per second per instance; a production version would need a faster language such as Go or Rust and a distributed vector store. The 4.20% false-positive rate may be too strict for cost-sensitive deployments, though Table 3 shows lower-threshold settings trade some accuracy for fewer false positives. Finally, strong performance on the unseen BeaverTails dataset suggests the results reflect real detection ability rather than pretraining memorization. Future work will also explore multimodal inputs and robustness against adaptive attackers who know the system in advance.

7 Conclusion

This paper presented Semantic Firewall, a layered, multi-agent defense-in-depth framework for securing LLM applications against prompt-injection and jailbreak attacks. By combining lightweight deterministic checks, an adaptive semantic memory, a parallel ensemble of eight specialized detectors, and a constrained LLM gate, Semantic Firewall achieves a 92.64% F1 score and 99.82% recall on the neuralchemy benchmark under cold-start evaluation, outperforming GPT-4o, Gemini 2.5 Flash, and Claude 3.5 Haiku. It resolves 66.88% of traffic without any LLM inference and cuts per-token API cost by roughly 3×. Cross-dataset results on BeaverTails, deepset, AI4Privacy, walledai, and multi-turn jailbreak benchmarks further confirm that the architecture generalizes beyond its primary evaluation setting, with an F1 score of 83.93% and recall of 87.59%. These results demonstrate that combining detection mechanisms with complementary failure modes and adaptive semantic memory offers a practical and scalable blueprint for securing production LLM-based applications against an evolving threat landscape.

Generative AI Usage Declaration

The authors used a large language model only to assist with manuscript preparation, including language editing and rephrasing for clarity, tone, and formality. All AI-assisted text was reviewed, verified, and edited by the authors, who take full responsibility for the accuracy, originality, and integrity of the manuscript.

References

1. Chen, J., et al.: A survey on prompt injection attacks in large language models. IEEE Trans. Inf. Forensics Secur. (2024)
2. OWASP: OWASP top 10 for large language model applications. OWASP Foundation (2023)
3. Greshake, K., et al.: Not what you've signed up for: compromising real-world LLM-integrated applications with indirect prompt injection. In: Proc. AISec (2023)
4. Liu, X., et al.: AutoDAN: generating stealthy jailbreak prompts on aligned large language models. arXiv preprint arXiv:2310.04451 (2023)

5. Chao, P., et al.: Jailbreaking black box large language models in twenty queries. arXiv preprint arXiv:2310.08419 (2023)
6. Wei, A., Haghtalab, N., Steinhardt, J.: Jailbroken: how does LLM safety training fail? In: Advances in Neural Information Processing Systems (NeurIPS) (2023)
7. Zou, A., et al.: Universal and transferable adversarial attacks on aligned language models. arXiv preprint arXiv:2307.15043 (2023)
8. Inan, M., et al.: Llama guard: LLM-based input-output safeguard for human-AI conversations. arXiv preprint arXiv:2312.06674 (2023)
9. Guardrails AI: Guardrails: validating and correcting the outputs of large language models. GitHub (2023)
10. Meta: PromptGuard: a safeguard model against prompt injections and jailbreaks (2024)
11. Li, H., Liu, X.: InjecGuard: benchmarking and mitigating over-defense in prompt injection guardrail models. arXiv preprint arXiv:2410.22770 (2024)
12. Kholkar, G., Ahuja, R.: CAPTURE: context-aware prompt injection testing and robustness enhancement. arXiv preprint arXiv:2505.12368 (2025)
13. Maiorano, A.C.: Evaluating prompt injection defenses for educational LLM tutors: security-usability-latency trade-offs. arXiv preprint arXiv:2605.06669 (2026)
14. ProtectAI: Rebuff: prompt injection defense. GitHub (2023)
15. Rebedea, T., et al.: NeMo guardrails: a toolkit for controllable and safe LLM applications. arXiv preprint arXiv:2310.10501 (2023)
16. Saltzer, J., Schroeder, M.: The protection of information in computer systems. Proc. IEEE **63**(9), 1278–1308 (1975)
17. Zhu, K., et al.: PromptBench: towards evaluating the robustness of large language models on adversarial prompts. arXiv preprint arXiv:2308.06588 (2023)
18. Robey, A., et al.: SmoothLLM: defending large language models against jailbreaking attacks. arXiv preprint arXiv:2309.02771 (2023)
19. Russinovich, M., Salem, A., Eldan, R.: Great, now write an article about that: the Crescendo multi-turn LLM jailbreak attack. arXiv preprint arXiv:2404.01833 (2024)
20. Reimers, N., Gurevych, I.: Sentence-BERT: sentence embeddings using Siamese BERT-networks. In: Proc. EMNLP-IJCNLP, pp. 3982–3992 (2019)
21. Chroma: ChromaDB: the AI-native open-source embedding database. GitHub (2023)
22. Malkov, Y., Yashunin, D.: Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. IEEE Trans. Pattern Anal. Mach. Intell. **42**(4), 824–836 (2020)
23. Shannon, C.E.: A mathematical theory of communication. Bell Syst. Tech. J. **27**(3), 379–423 (1948)
24. Wagner, R.A., Fischer, M.J.: The string-to-string correction problem. J. ACM **21**(1), 168–173 (1974)
25. McNemar, Q.: Note on the sampling error of the difference between correlated proportions or percentages. Psychometrika **12**(2), 153–157 (1947)
26. Neuralchemy: Prompt-injection-dataset. HuggingFace (2023), https://huggingface.co/datasets/neuralchemy/Prompt-injection-dataset
27. AI4Privacy: pii-masking-200k. HuggingFace (2023), https://huggingface.co/datasets/ai4privacy/pii-masking-200k
28. Walled.ai: Multi-Turn-Jailbreak. HuggingFace (2024), https://huggingface.co/datasets/walledai/Multi-Turn-Jailbreak

29. Ji, J., et al.: BeaverTails: towards improved safety alignment of LLM via a human-preference dataset. In: Advances in Neural Information Processing Systems (NeurIPS) (2023)
30. deepset: prompt-injections. HuggingFace (2023), https://huggingface.co/datasets/deepset/prompt-injections
31. Rajani, N., et al.: No Robots. HuggingFace (2023), https://huggingface.co/datasets/HuggingFaceH4/no_robots
32. OpenAI: GPT-4o system card. Tech. rep., OpenAI (2024), https://openai.com/index/gpt-4o-system-card/
33. Anthropic: Model card addendum: Claude 3.5 Haiku and upgraded Claude 3.5 Sonnet. Tech. rep., Anthropic (2024), https://assets.anthropic.com/m/1cd9d098ac3e6467/original/Claude-3-Model-Card-October-Addendum.pdf
34. Comanici, G., et al.: Gemini 2.5: pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities. arXiv preprint arXiv:2507.06261 (2025)
35. Meta: Llama Guard 4 model card. Meta AI (2025), https://github.com/meta-llama/PurpleLlama/blob/main/Llama-Guard4/12B/MODEL_CARD.md